

# Operating Systems

## Week 2: Processes & the Process API

### This week

- Process abstraction: what a process is and what state the OS tracks
- Address space basics: code/data/heap/stack; virtual vs physical
- Context switch idea: saving/restoring CPU state
- Process API: fork/exec/wait, exit status, zombies & orphans
- Shell as a case study: how a command actually runs

# Learning outcomes

What you should be able to explain & do by the end of this week

## Explain (concepts)

- Program vs process; a process = running program + state
- Why OS keeps a PCB (task\_struct) per process
- High-level context switch steps + why it has overhead
- What a virtual address space is (and why it isolates)
- Zombies vs orphans: definitions and causes

## Do (skills)

- Use ps/top to observe PID/PPID/state; interpret key columns
- Use /proc/<pid>/status, cmdline, maps, fd to inspect
- Write small programs using fork(), exec(), wait(), exit()
- Use strace to observe exec and wait behavior
- Explain how a shell uses fork+exec+wait

# The big idea: “a process is an illusion”

The OS makes one CPU and one memory look like many

## Illusion #1: CPU virtualization

Each process feels like it has a dedicated CPU.

How? Scheduler + context switching.

## Illusion #2: Memory virtualization

Each process feels like it owns private memory.

How? Address translation (page tables/Translation Lookaside Buffer (TLB) ).

## OS responsibilities (process perspective)

- Create/destroy processes
- Schedule CPU time fairly/efficiently
- Isolate processes (protection)
- Provide controlled sharing (files, sockets, IPC)
- Account & limit resources (CPU/memory/open files)

# Program vs process

Be precise with these terms

## Program

- Passive: bytes on disk (executable file)
- Code + static data
- Same program can run many times
- Example: `/bin/ls`, compiled “hello”

## Process

- Active: running program + state
- Has a PID; tracked by the kernel
- Own virtual address space
- CPU context: registers, PC, SP
- OS resources: open files, cwd, credentials

### Quick check (1 minute)

If you run “python script.py” in two terminals at the same time: one process or two?

Answer: two processes running the same program with different state.

# What state does the OS track for a process?

Think: what must be saved to stop and later resume

- CPU context: PC/IP, registers, stack pointer, flags
- Memory mappings: page tables (virtual pages → physical frames)
- Scheduling/accounting: priority, time slice, CPU time used
- I/O resources: open file descriptors, cwd, sockets
- Signals: pending signals + handlers
- Security: UID/GID, capabilities; access rights
- Relationships: PPID, children, process group/session

## Vocabulary

PCB (Process Control Block) is the kernel record storing this information.

In Linux, much of it lives in `task_struct` + memory-management structures.

# Process Control Block (PCB): conceptual view

Kernel data structure per process

## PCB fields (examples)

- PID, PPID, process group/session
- State (running/ready/blocked/zombie)
- Saved CPU registers + PC/SP
- Pointer to page tables
- Open file table pointer
- Scheduling params + accounting
- Pending signals / handlers
- Credentials (UID/GID)

## Kernel tracks many PCBs

Example snapshot (conceptual):

|       |              |     |
|-------|--------------|-----|
| PID 1 | init/systemd | [S] |
|-------|--------------|-----|

|          |      |     |
|----------|------|-----|
| PID 2210 | sshd | [S] |
|----------|------|-----|

|          |      |     |
|----------|------|-----|
| PID 5421 | bash | [R] |
|----------|------|-----|

|          |        |     |
|----------|--------|-----|
| PID 5440 | python | [S] |
|----------|--------|-----|

|          |     |     |
|----------|-----|-----|
| PID 5443 | top | [R] |
|----------|-----|-----|

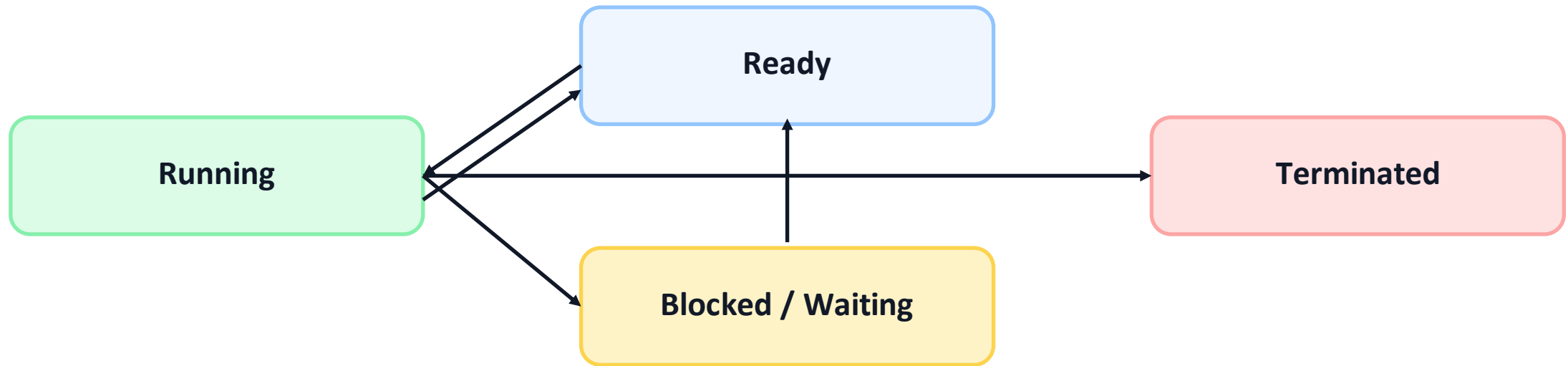
## How this appears in Linux

Many PCB fields are visible via `ps/top` and `/proc/<pid>/status`.

You rarely see the PCB directly, but you can observe its fields.

# Process states & transitions

Simplified model used in most OS textbooks



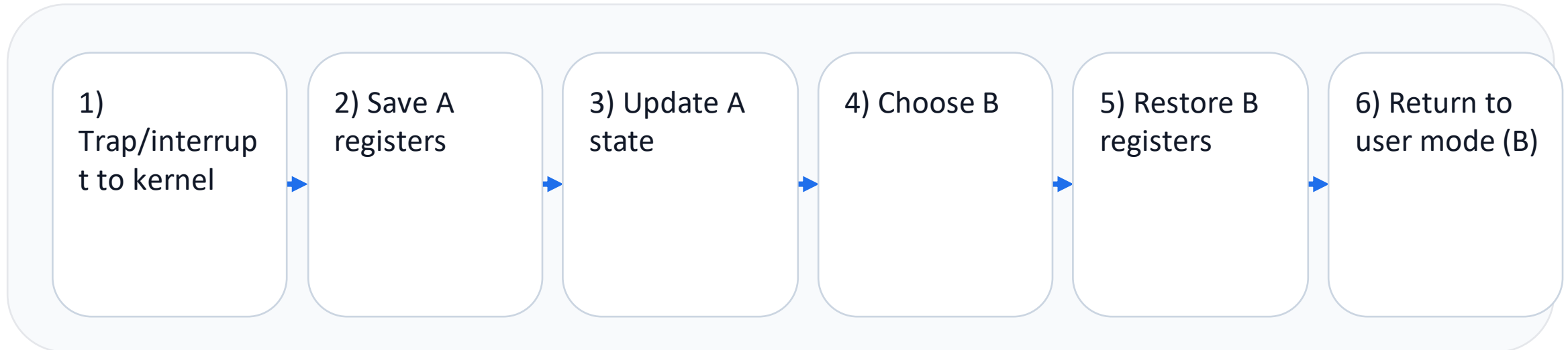
## Interpreting the arrows

- Ready → Running: scheduler dispatch
- Running → Ready: preempt (timer) or yield
- Running → Blocked: waiting for I/O/event
- Blocked → Ready: wakeup after event
- Running → Terminated: exit() or fatal signal

# Context switch: the core mechanism

Stop one process, resume another

- A context switch happens when the CPU stops executing process A and starts executing process B.
- The OS saves A's CPU context to A's PCB and restores B's saved context.
- Triggers: timer interrupt, blocking syscall (I/O), yield(), higher-priority task.



## Key takeaway

Context switching enables multitasking, but it costs time (save/restore + cache/TLB effects). OS design balances responsiveness vs overhead.

# Context switch overhead: what gets “cold”

Why switching too often hurts throughput

## Direct costs

- Saving/restoring registers
- Kernel book-keeping (PCB updates)
- Trap/interrupt entry and return
- Potential page-table switch

## Indirect costs (often bigger)

- CPU caches: working set displaced
- TLB misses after address-space switch
- Branch predictor disruption
- More memory traffic → slower execution

## Reasoning rule

Short slices → responsive but more overhead.

Long slices → efficient but may feel laggy for interactive tasks.

# Address space basics

A process sees virtual addresses

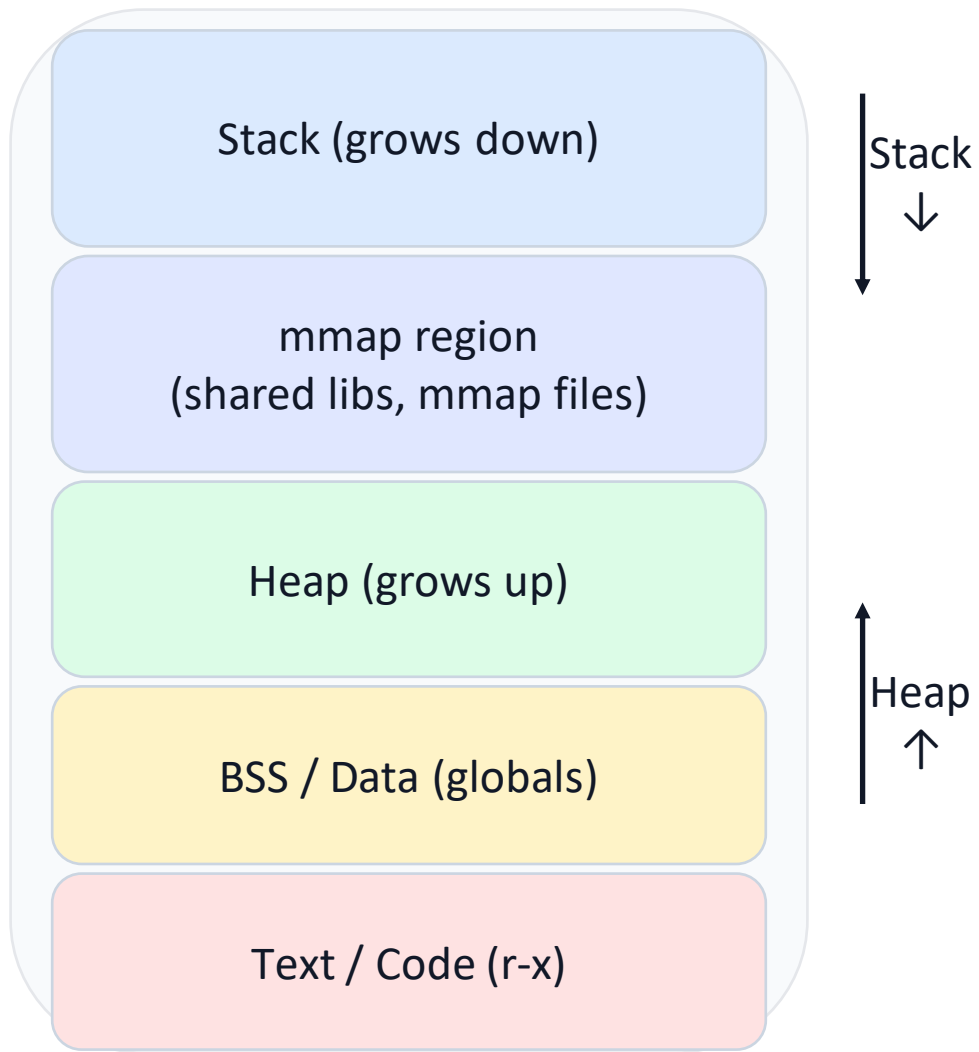
- CPU issues virtual addresses; hardware + OS translate to physical memory.
- Each process has its own virtual address space → strong isolation.
- Two processes can use the same virtual address but map to different frames.
- OS decides what is mapped: code, heap, stack, shared libs, mmap files.

## Why it's powerful

- Protection: one process cannot read/write another's memory
- Convenience: consistent layout per process
- Sharing: read-only pages can be shared
- Flexibility: physical memory can move without changing addresses

# Typical process memory layout (simplified)

From low → high virtual addresses



## Notes

- Real layouts vary (Address Space Layout Randomization (ASLR) randomizes base addresses).
- Stack: call frames (locals, return addr)
- Heap: dynamic allocations (malloc/new)
- mmap: shared libs + mapped files
- “r-x”: readable + executable

# Stack vs heap: common misconceptions

Useful for debugging & performance

## Stack

- Automatic (call/return)
- Fast allocation (move SP)
- Lifetime: until return
- Risk: stack overflow
- Per-thread stack

## Heap

- Explicit (malloc/free, new/delete, GC)
- Flexible lifetimes
- Overhead + fragmentation
- Risk: leaks, use-after-free (C)
- Shared by threads

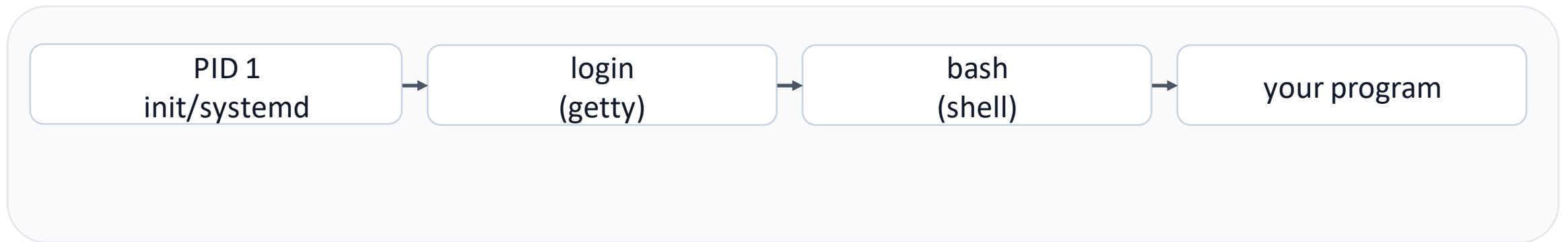
## OS connection

OS maps stack/heap pages and enforces protections, but your program must manage memory correctly.

# Process creation at system level

From boot to your terminal

- On boot, the kernel starts the first user-space process (traditionally init).
- On many distros, PID 1 is systemd (detail varies).
- PID 1 becomes the ancestor of most processes: services, login, shells, your programs.
- Parent/child relationships help interpret PPID and explain orphans.



## Try it

Run: `pstree -p | head`

Find your shell and observe children when you run commands.

# The Process API (Unix/Linux)

Four core operations you must master

## **fork()**

Create a new process (child).

*Why: New execution context.*

## **exec()**

Replace current process with a new program.

*Why: Run a different program.*

## **wait()/waitpid()**

Collect child exit status.

*Why: Synchronize; prevent zombies.*

## **exit()**

Terminate and return a status.

*Why: Signal success/failure.*

## **Mental model**

A shell typically does: `fork()` → (child) `exec()` → (parent) `wait()`

This pattern explains how most commands run.

# fork(): creating a child process

Return values, copy-on-write, and what is duplicated

- fork() creates a new process (child) that begins after the fork call.
- Return value: parent gets child PID; child gets 0; failure → -1.
- Child inherits: address space (logically), open files, environment, cwd.
- Copy-on-write: pages are shared until one side writes (efficient).

```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main() {
5     int x = 42;
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         x = x + 1;
10        printf("child: pid=%d, x=%d\n", getpid(), x);
11    } else {
12        printf("parent: pid=%d, child=%d, x=%d\n", getpid(), pid, x);
13    }
14 }
```

Question: do parent and child share x after fork? Why/why not?

# exec(): replacing the process image

The child becomes another program

- exec() does NOT create a new process. It transforms the current process into a new program.
- After a successful exec, old code/data/heap/stack are replaced.
- PID stays the same; open fds may remain unless close-on-exec is set.
- execve is core; execvp does PATH lookup.

```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main() {
5     char *args[] = {"ls", "-l", NULL};
6     printf("About to exec ls...\n");
7     execvp("ls", args);
8     perror("execvp failed");
9     return 1;
10 }
```

## Debugging tip

If code after exec() runs, exec failed. Always check errno (perror).

# wait() / waitpid(): collecting child termination

Why parents must reap children

- When a child finishes, the kernel keeps a record so the parent can read exit status.
- wait() blocks until any child exits; waitpid() can target a specific child.
- If parent never waits, child becomes a zombie until collected.
- waitpid can be non-blocking with WNOHANG (useful in shells).

```
1 #include <sys/wait.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main() {
6     pid_t pid = fork();
7     if (pid == 0) {
8         _exit(7);
9     } else {
10         int status;
11         waitpid(pid, &status, 0);
12         if (WIFEXITED(status))
13             printf("exit=%d\n", WEXITSTATUS(status));
14     }
15 }
```

# Exit status: communicating success/failure

Conventions and what the shell sees

## Conventions

- 0 usually means success
- Non-zero means error
- Shell exposes last status as \$? (bash)
- Commonly interpreted as 0–255

## Signals & abnormal termination

- May terminate by signal (SIGKILL, SIGSEGV, ...)
- wait status encodes exited vs signaled
- Use WIFSIGNALED/WTERMSIG
- Tools report “killed” or “segfault”

```
1 echo "hi"  
2 true; echo $?  
3 false; echo $?  
4
```

## Why it matters

Automation (scripts, CI, services) uses exit status to decide success/failure and trigger retries/alerts.

# Zombies: “dead but not gone”

Terminated child awaiting collection by its parent

- Zombie = finished execution but still has an entry in the process table.
- Reason: OS keeps exit status until parent calls `wait()`.
- Zombies do not consume CPU, but consume PID/table resources.
- Many zombies → resource exhaustion.

## How to spot zombies

`ps -el` (look for state Z)

`ps -o pid,ppid,state,cmd -p <pid>`

In top, zombies may show as “Z”.

## How to fix

In code: parent must `wait/waitpid`.

If parent is buggy, restarting it may be required.

If parent exits, zombies are adopted and reaped by PID 1.

# Orphans: “parent died first”

What happens when the parent exits

- Orphan = running process whose parent has terminated.
- OS re-parents orphans to PID 1 so every process has a parent.
- This ensures the orphan will be reaped when it exits.
- Orphans are not inherently bad; daemons often detach intentionally.

```
1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main(){
5     pid_t pid=fork();
6     if(pid==0){
7         sleep(2);
8         printf("child ppid=%d\n", getppid());
9     } else {
10        printf("parent exiting...\n");
11        _exit(0);
12    }
13 }
```

Exercise: run, then observe PPID changes (ps -o pid,ppid,cmd -p <pid>).

# Shell as a case study

A shell is a user program that uses the Process API

- Shell loop: read → parse → execute → (optionally) wait.
- Built-ins (cd, exit, export) must run inside the shell process.
- External commands: fork child, then exec in child.
- Advanced shells add pipelines, redirection, job control, signals.

## Pseudo-code of a simple shell

```
while (true) {  
    prompt();  
    line = readLine();  
    argv = parse(line);  
    if (builtin(argv)) runBuiltin(argv);  
    else {  
        pid = fork();  
        if (pid==0) execvp(argv[0], argv);  
        else waitpid(pid, ...);  
    }  
}
```

# How “ls -l” runs (step-by-step)

Fork → exec → wait

1. Shell parses: `argv = ["ls", "-l"]`

2. Shell calls `fork()`

3. Child calls `execvp("ls", argv)`

4. Kernel loads `/bin/ls` into child address space

5. Child runs and calls `exit()`

6. Parent calls `waitpid()` and continues

## Try it (2 terminals)

A) `ps -o pid,ppid,state,cmd -C bash`

B) run “ls -l” repeatedly and observe short-lived children.

# Shell preview: fds (0/1/2), redirection, pipes

Processes + I/O are connected via file descriptors

- Every process starts with: 0=stdin, 1=stdout, 2=stderr.
- Redirection changes where fds point: stdout→file, stderr→file.
- Pipes connect stdout of one process to stdin of another.
- Shell sets up fds BEFORE exec(); program then uses them normally.

```
1 # stdout to file
2 ls -l > out.txt
3
4 # stderr to file
5 find / -name secret 2> errors.txt
6
7 # pipe
8 cat out.txt | grep ".pdf"
9
```

# Linux: observe real processes

ps/top and /proc show you process state

## Commands you must know

- `ps aux | head`
- `ps -o pid,ppid,state,etime,cmd -p <pid>`
- `top / htop`
- `pstree -p`
- `kill -SIGTERM <pid>`

## /proc: process filesystem

- `/proc/<pid>/status`
- `/proc/<pid>/cmdline`
- `/proc/<pid>/maps`
- `/proc/<pid>/fd/`
- Tip: `cat /proc/self/status`

### Mini-task (10 min)

Run: `sleep 1000`

Inspect `/proc/<pid>/status` and record: PID, PPID, State, VmRSS.

# strace: watch exec and wait in real time

A microscope for system calls

- strace prints system calls made by a program (and their results).
- Observe `execve`, `wait*`, `clone/fork` (implementation detail), `exit_group`.
- Use it to connect abstract APIs to real OS behavior.

```
1 # Trace a command (including children)
2 strace -f -o trace.txt ls -l
3
4 # Focus on process-related syscalls
5 strace -f -e trace=process ls
6
7 # Inspect
8 grep -E "execve|fork|clone|wait|exit" trace.txt | head
9
```

## How to read a line

`syscall(args...) = return`

If return is -1, strace shows `errno` (reason).

# In-class exercises

Short tasks to build intuition

## Exercise A: process-tree reasoning (7 min)

Parent forks child; child forks grandchild; child exits; grandchild sleeps.

Q1) PID/PPID over time?

Q2) When does grandchild become orphan?

Q3) Who reaps it when it exits?

## Exercise B: inspect /proc (10 min)

Run: `sleep 1000`

Find PID. Inspect `status/cmdline/fd`.

Write 3 observations + meaning.

## Exercise C: strace interpretation (10–12 min)

Run: `strace -f ls -l`

Identify `execve`, `openat`, `exit_group`.

Explain in 5–6 sentences what happened.

# Homework / practice set (3–4 hours)

Basic → intermediate

## Concept questions

- 1) Why OS keeps a zombie entry after child exits?
- 2) What is preserved across exec? What is replaced?
- 3) Why built-ins like `cd` cannot be external exec?
- 4) Define PID, PPID, process group, session.
- 5) Give one direct and one indirect context-switch cost.

## Coding / tooling tasks

- 6) Fork N children and wait for all.
- 7) Create zombies temporarily and then fix it.
- 8) Print cmdline of a PID via `/proc`.
- 9) Compare syscalls of `ls` vs `echo` using `strace`.
- 10) Optional: timeout—kill child if it runs too long.

## Submission

Submit answers + screenshots + short explanations. Include an “AI usage note” if used.

# Lab 1: Mini-shell v1 (fork/exec/wait)

Hands-on assignment

## Goal

Implement a simple shell that runs external programs using `fork()+exec()+wait()`.

Focus: correctness + understanding.

## Required (core)

- Prompt loop (myshell\$)
- Tokenize into argv
- `execvp` external commands
- `wait` + show exit status
- Built-ins: `cd`, `exit`
- Handle invalid commands

## Bonus (choose 1–2)

- Background & (no wait)
- Reap with `WNOHANG`
- Basic redirection: `>` and `<`
- Better parsing (quotes)
- Simple history

## Deliverables & grading (example)

- Code + README (build/run)
- 1–2 page report: design + what you learned

Rubric: correctness 50%, robustness 20%, clarity 15%, explanation 15% (oral spot-check possible).

# Lab test / assessment checklist

How we will check understanding

## Automated checks (examples)

- Runs: ls, pwd, echo
- cd works (verified by pwd)
- Captures non-zero exit codes
- No zombies after repeated runs
- Handles invalid command gracefully

## Oral/viva (examples)

- Explain fork return values
- Why exec in child (not parent)
- What happens if parent doesn't wait
- Show /proc evidence (PID/PPID/state)
- Interpret a short strace snippet

## Common pitfalls

argv not NULL-terminated; exec failure not handled; parent calling exec; not reaping children; brittle parsing.

# Wrap-up & exit ticket

## Summary

- Process = running program + OS-tracked state
- Context switch enables CPU virtualization
- Address space enables isolation via virtual memory
- fork creates child; exec replaces program; wait collects status; exit returns code
- Zombies happen if parent doesn't wait; orphans are adopted

## Exit ticket

- 1) In one sentence: what does exec() do?
- 2) What is a zombie and how do you prevent it?
- 3) Run: `strace -f -e trace=process ls`  
Paste 3 lines and label parent vs child.

**Next week preview: CPU scheduling policies and performance tradeoffs.**